1. In "Compute multipliers" and "Update the remainder of the matrix", the indexes "i" and "j" are initialized such that the operations are done simultaneously without an explicit "for loop". The vectorized operations are "A(i,k) = A(i,k)/A(k,k);" and "A(i,j) = A(i,j) - A(i,k)*A(k,j);".
2.
    a. From backslashtx.m, I modified the variable 'b' to be an identity matrix and put a loop over forward elimination/backword substitution function calls.

```
function x = myinv(A)
% myinv  Find inverse for A based on backslashtx.m
% which is x = bslashtx(A,b) solves A*x = b.
% x = myinv(A) solves A*x = I where I is the identity matrix
% and x denotes A^-1.
[n,n] = size(A);
b = eye(n); % b is an identity matrix of size n*n where n is
size(A,1).
if isequal(triu(A,1),zeros(n,n))
   % Lower triangular
   x = zeros(n);
   for i = 1:n
        x(:,i) = forward(A,b(:,i));
   end
   return
elseif isequal(tril(A,-1),zeros(n,n))
   % Upper triangular
   x = zeros(n);
   for i = 1:n
        x(:,i) = backsubs(A,b(:,i));
   end
   return
elseif isequal(A,A')
   [R,fail] = chol(A);
   if ~fail
      % Positive definite
      y = zeros(n); % add a loop for each column x_j.
      for i = 1:n
            y(:,i) = forward(R',b(:,i));
      end
      x = zeros(n); % add a loop for each column x_j.
      for i = 1:n
            x(:,i) = backsubs(R,y(:,i));
      end
      return
   end
end
```

```
% Triangular factorization
[L,U,p] = lutx(A);

% Permutation and forward elimination
y = zeros(n); % add a loop for each column x_j.
for i = 1:n
    y(:,i) = forward(L,b(p,i));
end
% Back substitution
x = zeros(n); % add a loop for each column x_j.
for i = 1:n
    x(:,i) = backsubs(U,y(:,i));
end
end
% ------------------------------

function x = forward(L,x)
% FORWARD. Forward elimination.
% For lower triangular L, x = forward(L,b) solves L*x = b.
[n,n] = size(L);
x(1) = x(1)/L(1,1);
for k = 2:n
   j = 1:k-1;
   x(k) = (x(k) - L(k,j)*x(j))/L(k,k);
end
end

% ------------------------------

function x = backsubs(U,x)
% BACKSUBS.  Back substitution.
% For upper triangular U, x = backsubs(U,b) solves U*x = b.
[n,n] = size(U);
x(n) = x(n)/U(n,n);
for k = n-1:-1:1
   j = k+1:n;
   x(k) = (x(k) - U(k,j)*x(j))/U(k,k);
end
end
```

b. I made a matrix ‘A’ by running qr on a random matrix ‘a’ of size 4*4. Then, I compared the result of myinv(A) and inv(A) which appears to be the same.

```
>> A

A =

   -1.6775   -0.6771   -1.5305   -1.0280
    0.0596   -0.7846   -0.5798   -0.5348
    0.3673   -0.1881   -0.3720   -0.4389
    0.3622    0.0004   -0.3479    0.7210

>> A_prime = myinv(A)

A_prime =

   -0.3147    0.0149    1.0712    0.2144
    0.3152   -1.9580    1.7161    0.0417
   -0.4183    0.6451   -1.1866   -0.8403
   -0.0439    0.3050   -1.1118    0.8737

>> inv(A)

ans =

   -0.3147    0.0149    1.0712    0.2144
    0.3152   -1.9580    1.7161    0.0417
   -0.4183    0.6451   -1.1866   -0.8403
   -0.0439    0.3050   -1.1118    0.8737

fx >>
```

The next test case is an upper triangular matrix ‘U’ which was made by running lu on a random matrix ‘u’ of size 4*4. Then, I compared the result of myinv(U) and inv(U) which appears to be the same.

```
>> U

U =

    0.7577    0.1712    0.0462    0.3171
         0    0.5382    0.0519    0.6392
         0         0    0.8050   -0.0623
         0         0         0    0.0611

>> U_prime = myinv(U)

U_prime =

    1.3197   -0.4198   -0.0487   -2.5074
         0    1.8582   -0.1197  -19.5678
         0         0    1.2422    1.2659
         0         0         0   16.3710

>> U_prime = inv(U)

U_prime =

    1.3197   -0.4198   -0.0487   -2.5074
         0    1.8582   -0.1197  -19.5678
         0         0    1.2422    1.2659
         0         0         0   16.3710

fx >>
```

The next test case is a lower triangular matrix ‘L’ which was made by running lu on a random matrix ‘l’ of size 4*4. Then, I compared the result of myinv(L) and inv(L) which appears to be the same.

```
>> L

L =

    1.0000         0         0         0
    0.9807    1.0000         0         0
    0.5176   -0.1055    1.0000         0
    0.8650    0.2394    0.7981    1.0000

>> L_prime = myinv(L)

L_prime =

    1.0000         0         0         0
   -0.9807    1.0000         0         0
   -0.6211    0.1055    1.0000         0
   -0.1346   -0.3236   -0.7981    1.0000

>> L_prime = inv(L)

L_prime =

    1.0000         0         0         0
   -0.9807    1.0000         0         0
   -0.6211    0.1055    1.0000         0
   -0.1346   -0.3236   -0.7981    1.0000

fx >>
```

For the last test case, I made a symmetric matrix ‘S’ of size 4*4 by using the function symdec. Then, I compared the result of myinv(S) and inv(S) which appears to be the same.

```
>> S

S =

     5     6     8    11
     6     7     9    12
     8     9    10    13
    11    12    13    14

>> S_prime = myinv(S)

S_prime =

  -15.5000   16.5000   -0.5000   -1.5000
   16.5000  -18.5000    1.5000    1.5000
   -0.5000    1.5000   -1.5000    0.5000
   -1.5000    1.5000    0.5000   -0.5000

>> S_prime = inv(S)

S_prime =

  -15.5000   16.5000   -0.5000   -1.5000
   16.5000  -18.5000    1.5000    1.5000
   -0.5000    1.5000   -1.5000    0.5000
   -1.5000    1.5000    0.5000   -0.5000

fx >>
```

3. I used a scalar sig as a counter that counts the times of permutation in ‘swap pivot row’ and output the value as 1 or -1 according to the modulo ‘mod(sig, 2)’.

```
function [L,U,p,sig] = lutx(A)
%LU Triangular factorization
% [L,U,p,sig] = lutx(A) computes a unit lower triangular
% matrix L, an upper triangular matrix U, a permutation
% vector p, and a scalar sig, so that L*U = A(p,:) and
% sig = +1 or -1 if p is an even or odd permutation.

[n,n] = size(A);
p = (1:n)';
sig = 0; % permutation counter
for k = 1:n-1

   % Find index of largest element below diagonal in k-th
column
   [r,m] = max(abs(A(k:n,k)));
   m = m+k-1;

   % Skip elimination if column is zero
   if (A(m,k) ~= 0)

      % Swap pivot row
      if (m ~= k)
         A([k m],:) = A([m k],:);
         p([k m]) = p([m k]);
         sig = sig + 1; % count the number of permutation
      end

      % Compute multipliers
      i = k+1:n;
      A(i,k) = A(i,k)/A(k,k);

      % Update the remainder of the matrix
      j = k+1:n;
      A(i,j) = A(i,j) - A(i,k)*A(k,j);
   end
end

% Separate result
L = tril(A,-1) + eye(n,n);
U = triu(A);
if mod(sig,2) == 1 % check whether the counter is odd or even
    sig = -1; % assign appropriate value
else
    sig = 1;
end
```

4. The function ‘mydet.m’ computes the determinant of the matrix ‘A’ by performing LU decomposition on ‘A’ to get the upper triangular matrix ‘U’ and ‘sig’. Then, ‘sig’ and the values along the diagonal of ‘U’ are multiplied to get the determinant of ‘A’.

```
function det_A = mydet(A)
% MYDET find the determinant of a matrix A.
% det_A = mydet(A)uses the modified lutx to compute the
determinant of A.
% from det(A) = sig * set(U).
[~,u,~,sig] = lutx(A); % get U and sig from lutx
det_A= sig; % start with sig
for i = 1:size(u,1) % \prod_i{U_ii} * sig
    det_A = det_A * u(i,i);
end
end
```

I made a matrix ‘A’ by running qr on a random matrix ‘a’ of size 4*4. Then, I compared the result of mydet(A) and det(A) which appears to be the same.

```
>> A

A =

   -1.6775   -0.6771   -1.5305   -1.0280
    0.0596   -0.7846   -0.5798   -0.5348
    0.3673   -0.1881   -0.3720   -0.4389
    0.3622    0.0004   -0.3479    0.7210

>> mydet(A)

ans =

   -0.6883

>> det(A)

ans =

   -0.6883

fx >>
```

The next test case is an upper triangular matrix ‘U’ which was made by running lu on a random matrix ‘u’ of size 4*4. Then, I compared the result of mydet(U) and det(U) which appears to be the same.

```
>> U

U =

    0.7577    0.1712    0.0462    0.3171
         0    0.5382    0.0519    0.6392
         0         0    0.8050   -0.0623
         0         0         0    0.0611

>> mydet(U)

ans =

    0.0201

>> det(U)

ans =

    0.0201

fx >>
```

The next test case is a lower triangular matrix ‘L’ which was made by running lu on a random matrix ‘l’ of size 4*4. Then, I compared the result of mydet(L) and det(L) which appears to be the same as ‘1’.

```
>> L

L =

    1.0000         0         0         0
    0.9807    1.0000         0         0
    0.5176   -0.1055    1.0000         0
    0.8650    0.2394    0.7981    1.0000

>> mydet(L)

ans =

     1

>> det(L)

ans =

     1

fx >>
```

For the last test case, I made a symmetric matrix ‘S’ of size 4*4 by using the function symdec. Then, I compared the result of mydet(S) and det(S) which appears to be the same.

```
>> S = symdec(4,4)

S =

     5     6     8    11
     6     7     9    12
     8     9    10    13
    11    12    13    14

>> mydet(S)

ans =

   -2.0000

>> det(S)

ans =

   -2.0000

>>
```